第 0-2 讲 · 装好你的工作台

> **模块 0 · 热身** ｜ 建议用时 **40 分钟**（阅读 20 + 动手 20）
> 前置要求：完成第 0-1 讲（会用 `cd` 、 `ls` 、 `pwd` ）
> 本讲结束时，你会看到记账簿的窗口在自己手里打开——**这是整个课程第一个真正的成就。**

本节目标

读完并做完这一讲，你能：

1. **说出"跑起来"到底需要哪三样东西**（Node.js、依赖、启动命令），以及缺了哪一样会报什么错。
2. 自己检查电脑上有没有装 Node.js，并**看懂版本号的含义**。
3. 理解 `node_modules` 是什么、为什么它必须存在、为什么可以删掉重建。
4. **亲手启动记账簿的开发模式**（ `npm run dev` ），看到窗口出现。
5. 知道"窗口是白的""报错了""卡住了"分别该去看什么。

一、先讲清楚：让一个项目"跑起来"需要什么

1.1 现实比喻：开一家餐厅

把记账簿想象成一家餐厅。要让它开门营业，你需要：

需要的东西	对应到项目	缺了会怎样
一个厨房（会做菜的环境）	**Node.js**	什么命令都不认识，终端直接报"无法识别"
食材和厨具（菜谱里写的材料）	**依赖（ `node_modules` ）**	找不到模块，报 `Cannot find module`
开门的指令（今天做什么菜）	**启动命令**（ `npm run dev` ）	你不说，它就静静躺着

这三样缺一不可，而且顺序固定：先有厨房 → 再备食材 → 才能开火。

大部分人第一次失败，都是因为**跳过第二步**（没装依赖就直接跑启动命令），然后被一堆红色报错吓退。理解了这张表，你就知道该按什么顺序排查。

1.2 为什么"开发模式"和"平时打开软件"不一样

你平时双击 `记账簿.exe` 就能用。那为什么学开发还要跑命令？

因为**那是一个"成品"**——所有代码已经被打包、压缩、装进一个盒子里，你改不了它。

而 `npm run dev` 打开的是**"活的版本"**：

	双击 exe	`npm run dev`
用的是哪份代码	打包好的旧副本	**你正在改的源码**
改一行代码	没反应	**窗口自动刷新**
能看到报错细节	看不到	**能看到完整报错**
用途	日常记账	开发、调试

关键：开发模式下你改代码，窗口会立刻变。这就是为什么后面几讲要改功能，必须在开发模式里做。

二、概念：Node.js 到底是什么

2.1 一句话解释

Node.js 是一个"能在你的电脑上运行 JavaScript 的程序"。

更准确地说：JavaScript 原本只能在浏览器里跑（网页上的按钮、动画）。有人把它从浏览器里"拿出来"，做成了一个独立程序，于是它就能像其它程序一样读写文件、开窗口、连数据库。这个东西就叫 Node.js。

为什么你的项目需要它：

- 记账簿的代码是用 JavaScript（严格说是 TypeScript，稍后会讲）写的；
- 要**运行**这些代码 → 需要 Node.js；
- 要**打包**成 exe → 也需要 Node.js 的工具链。

2.2 npm 是什么

装 Node.js 的时候，会**顺带装上一个叫 npm 的东西**。

`npm` = **N**ode **P**ackage **M**anager（Node 包管理器）。它的工作只有两件：

1. **装依赖**：`npm install` —— 照着清单把需要的零件下载下来，放进 `node_modules`
2. **跑脚本**：`npm run xxx` —— 执行 `package.json` 里预先写好的命令

你之前看到的所有 `npm run ...`，都是在做第 2 件事。

2.3 版本号怎么读

```
v24.18.0
 ↑  ↑  ↑
 |  |  └─ 修订号：修 bug，不改功能
 |  └──── 次版本号：加了新功能，但兼容老代码
 └─────── 主版本号：有重大变化，可能不兼容
```

关于"要装哪个版本"：这个项目的 `package.json` 里**没有**写死 Node 版本要求（没有 `engines` 字段），所以没有硬性门槛。实际经验是：**Node 18 以上都能跑**，开发机用的是 24。

更重要的原则：你不需要追求最新版。 能跑就别动它——这是新手最容易犯的错：为了"新"去升级环境，结果项目跑不起来。等你真的遇到"必须升级"的报错，再升也不迟。

2.4 `node_modules` 到底是什么，为什么那么大

`package.json` 里写着这样一段（这是**清单**）：

```
"dependencies": {
  "better-sqlite3": "12.4.1",
  "onnxruntime-node": "^1.30.0"
}
```

意思是："我需要别人写好的两个零件，名字和版本在这。"

`npm install` 会照着这张单子去下载，**而且不只是这两个**——这两个零件自己还依赖别的零件，那些又依赖别的……最后滚成一棵大树。这就是为什么 `node_modules` 里有**一万多个文件、接近 1 GB**，而你自己只写了几十个文件。

三条必须记住的规矩：

1. **永远不要手动改 `node_modules` 里的任何文件。**你改了，下次 `npm install` 就被覆盖，白改。
2. `node_modules` **可以整个删掉重建**（`npm install` 就能恢复）。它不是你的成果，是下载来的材料。
3. **它不能直接复制到别的电脑/别的盘。**里面有些零件（如 `better-sqlite3`）是**针对本机编译**的，路径写死了；搬过去大概率加载失败，必须重新装。

三、在项目里看真实文件

打开 `C:\Users\34123\Desktop\记账簿\package.json`（用记事本或 VS Code），你会看到三类关键内容：

3.1 身份信息

```
"name": "daily-ledger",
"version": "0.1.0",
"description": "..."
```

这就是为什么安装包叫 `daily-ledger-0.1.0-portable.exe`——名字和版本都从这里来。**改这里，打包出来的文件名就变。**

3.2 依赖清单

```
"dependencies": { ... },      ← 运行时需要的（打包进安装包）
"devDependencies": { ... }    ← 只在开发时需要的（不打包）
```

两者区别很重要：

- `dependencies`：软件**运行时**要用的，比如数据库、界面库 → **会进安装包**
- `devDependencies`：只有**开发和打包**时用的，比如 TypeScript、打包工具 → **不进安装包**

所以你发给朋友的安装包是 116 MB，而项目目录有 3 GB——大量东西是不发出去的。

3.3 命令清单（scripts）

```
"scripts": {
  "dev": "electron-vite dev",
  "build": "electron-vite build",
  "typecheck": "...",
  "selftest": "...",
  ...
}
```

这是你以后最常看的一段。 `npm run dev` 里的 `dev`，就是这里的键名。你可以自己加一条，比如：

```
"hello": "echo 你好，这是我的第一条命令"
```

存盘后在终端执行 `npm run hello`，就会打印那句话。**这是本讲动手任务的第一步**，先建立"我能指挥它"的感觉。

四、动手任务

步骤 1：检查厨房（Node.js 在不在）

```
cd "C:\Users\34123\Desktop\记账簿"
node -v
npm -v
```

预期看到：两个版本号（如 `v24.18.0` / `10.9.0`）。

如果报"无法将 node 识别为……"：去 <https://nodejs.org> 下载 **LTS 版**（长期支持版，更稳），一路下一步装完，**关掉终端重新打开**（重要：环境变量要重开终端才生效），再试一次。

步骤 2：加一条自己的命令（建立指挥感）

用记事本或 VS Code 打开 `package.json`，在 `"scripts"` 里加一行（注意上一行末尾要有逗号）：

```
"hello": "echo 你好，我是记账簿的主人"
```

存盘，然后：

```
npm run hello
```

预期看到：终端打印出 `你好，我是记账簿的主人`。

这一步的意义：你已经修改了项目的一个文件，并且它按你的意思执行了。**这是"我能动它"的第一个证据。**

步骤 3：备食材（安装依赖）

先看现在有没有：

```
Test-Path node_modules
```

- 输出 `True` → 已经有了，**跳过安装**（别浪费十分钟重装）。
- 输出 `False` → 执行安装：

```
npm install --ignore-scripts
```

为什么带 `--ignore-scripts`：有些零件（比如数据库引擎）安装时会尝试自己编译，这在 Windows 上经常失败并中断整个安装。加这个参数让它**跳过自动编译**——项目里已经准备好了编译好的版本，不需要它现场编。

> 顺带说：这条命令就写在项目的 `README.md` 里（"装依赖"那一节）。**遇到不确定该敲什么命令时，先去看 `README.md`** ——这是项目作者留给你的说明书，比搜索引擎靠谱。

预期看到：进度条滚动，最后是 `added XXX packages`。

这一步可能很慢（5–15 分钟，取决于网速），因为它要下载将近 1 GB 的东西。**耐心等它跑完，不要中途关窗口。**

步骤 4：开火（启动开发模式）

```
npm run dev
```

预期看到：

1. 终端开始输出编译信息（几秒到半分钟）；
2. 然后**弹出一个应用窗口**，标题是「记账簿」；
3. 窗口里是你熟悉的界面——但这是**从源码跑起来的活的版本**。

保持这个窗口不要关。这个终端就是记账簿的"生命线"，关掉它软件就退出了。

步骤 5：验证"活"的感觉（本讲高潮）

不要关终端，另外用记事本/VS Code 打开：

```
C:\Users\34123\Desktop\记账簿\src\renderer\src\layouts\AppLayout.tsx
```

找到导航列表（搜 `NAV`），你会看到类似：

```
{ to: '/records', label: '记录', icon: 'records' },
```

把 `'记录'` 改成 `'记录（我改的）'`，存盘。

回头看窗口——它应该**自动刷新**，侧栏那一项变成了「记录（我改的）」。

这就叫"活的版本"。 你刚才完成了人生第一次真正意义上的代码修改。（改完记得改回去，或者留着也行，不影响功能。）

步骤 6：收工

回到终端，按 `Ctrl + C`，窗口会关闭，开发模式停止。

注意：`Ctrl+C` 是"中断当前命令"，不是关终端窗口。养成习惯：跑完就停掉，别让一堆进程堆着。

卡住了怎么办

现象	原因	怎么办
`无法将 node 识别为…`	没装 Node，或装完没重开终端	装 LTS 版，**关掉终端重新打开**
`npm install` 卡住不动很久	在下载，网络慢；或卡在某个包的编译	等 15 分钟；还不动就 `Ctrl+C` 后重试一次
`Cannot find module 'xxx'`	依赖没装全	重新 `npm install --ignore-scripts`
窗口全白，什么都没有	前端还没编译完，或渲染出错	等 10 秒；还白就看终端里的报错（下一讲会讲）
改了代码没反应	改的不是正在跑的文件 / 没存盘	确认存盘；确认改的是 `src` 里的文件
终端提示端口被占用	上次的 dev 还开着	关掉所有相关终端/窗口，重开
提示 `EACCES` / 权限错误	项目放在受保护目录	项目在桌面，一般不会；真遇到就重启终端

五、自测题

1. 让项目跑起来需要哪三样东西？按什么顺序？

2. `npm run dev` 和双击 `记账簿.exe` 有什么区别？为什么开发时要用前者？

3. `node_modules` 有 1 GB，但它不是你的成果。为什么这么说？它能不能删？

4. `dependencies` 和 `devDependencies` 的区别是什么？哪个会进安装包？

5. 你在 `package.json` 里加了 `"hello": "echo 你好"`，之后该执行什么命令？

答案与解析

1. ① **Node.js**（运行环境，"厨房"）→ ② **依赖** `node_modules`（材料）→ ③ **启动命令**（开火）。 → 顺序不能乱。最常见的新手错误是跳过 ②，直接跑 ③，然后被 `Cannot find module` 淹没——其实只是没备料。

2. 双击 exe 用的是**打包好的旧副本**，改代码没反应；`npm run dev` 用的是**你正在改的源码**，改一行、存盘，窗口自动刷新。 → 所以**开发必须在 dev 模式下进行**。另外 dev 模式能看到完整报错，这是排查问题的唯一依据。

3. 因为里面全是从网上下载的**第三方零件**，你一行都没写。它由 `package.json` 的依赖清单 + `npm install` 随时重建。 → **可以整个删掉重建**。但**不能直接复制到别处**——部分零件是针对本机编译的，路径写死，搬家会加载失败。要用就重新 `npm install`。

4. `dependencies` 是**运行时**需要的（数据库、界面库）→ **会进安装包**；`devDependencies` 只在**开发和打包**时用（TypeScript、打包工具）→ **不进安装包**。 → 这解释了"项目 3 GB，安装包只有 116 MB"。

5. `npm run hello`。 → 规律：`package.json` 的 `scripts` 里叫什么，就 `npm run 什么`。这个键名就是你调用它的"名字"。

六、术语表

术语	一句话解释
Node.js	能在电脑上运行 JavaScript 的程序，项目的运行环境
npm	Node 自带的包管理器：装依赖、跑脚本
依赖 / Dependency	别人写好、你的项目要用的代码库
`node_modules`	所有依赖存放的文件夹，可删可重建，不可搬移
`package.json`	项目的身份证：名字、版本、依赖清单、命令清单
`package-lock.json`	依赖的精确版本记录，保证每次装出来的版本一致
LTS	Long Term Support（长期支持版），装 Node 选这个更稳
开发模式 / dev	从源码直接运行、改代码即时生效的运行方式
`npm install`	照着清单把所有依赖下载下来
`--ignore-scripts`	安装时跳过依赖自己的编译脚本，避免 Windows 上编译失败
`npm run xxx`	执行 `package.json` 里名叫 xxx 的那条命令
`Ctrl + C`	中断终端里正在运行的命令
`dependencies`	运行时依赖，会打进安装包
`devDependencies`	开发时依赖，不进安装包

七、本讲与项目的对应

- 你改动的文件：`package.json`（加了 `hello` 命令）、`AppLayout.tsx`（改了导航文字，可还原）
- **不会影响你的任何记账数据** —— 数据在 `<你的数据目录>`，开发模式默认用沙箱/当前配置，本讲未涉及数据库写操作
- 上一讲：0-1 终端与命令行 ｜ 下一讲：**1-1 双击之后发生了什么**（进入模块 1，开始真正理解程序）

讲义版本：v1 ｜ 对应项目版本：0.1.0